UNIVERSITY OF MINES AND TECHNOLOGY

TARKWA, GHANA

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEM

LABORATORY REPORT

TOPIC TITLE: Testing and Documenting Password Cracking Techniques Against Lab Hash Dumps

TEAM: Red Team
COURSE CODE: CY 375
STUDENT NAME: Amiratu Ewurabena Sarkodie Abubakari
INDEX NUMBER: FCM.41.018.237
DATE: August 3, 2026

TABLE OF CONTENT

Abstract

This report documents a hands-on cybersecurity lab project testing how password cracking tools work against different types of password setups. Using Kali Linux and Hashcat, five common attack methods were tested: basic dictionary attacks, mask attacks (guessing patterns), rule-based attacks (guessing common variations), hybrid attacks (combining words and numbers), and testing a stronger hashing formula (SHA-256).

The test results showed that simple passwords inside text files get cracked almost instantly. Tricks that users like to use to make their passwords feel safe—such as capitalizing the first letter or adding 1! at the end (like Monkey1!)—fail easily when using rule sets. The report ends with simple advice for building safer password systems, like using longer passphrases and using modern password hashing methods instead of old ones like MD5.

INTRODUCTION

Background

Websites and operating systems do not save user passwords in plain text. Instead, they run them through a math formula called a hash function and store the scrambled result (a hash).

When a hash function works properly, it follows three main rules:

1. You cannot easily turn a hash back into the original word.
2. Two different passwords should not produce the exact same hash.
3. You cannot easily find two words that match the same hash output.

Even with these rules, if an attacker gets a copy of a database with stored hashes, they can run an offline password cracking attack. They use fast computers to hash millions of candidate words every second until they find one that matches the target hash.

Project Goals

The goal of this project is to act like a Red Team security researcher and test how weak passwords fall apart under different attack methods.

Specifically, this project sets out to:

- Automate hash creation using a simple Bash script.
- Test five different Hashcat attack modes to see how fast they crack passwords.
- Compare an older, faster hashing style (MD5) against a newer one (SHA-256).
- Give clear tips on how system administrators can better protect user accounts.

Tools and Frameworks Used

Industry Standards

This lab was completed based on standard real-world guidelines:

- MITRE ATT&CK (Technique T1110.002): Covers offline password cracking techniques used by attackers after downloading stored password files.
- NIST Password Guidelines (SP 800-63B): Advises against forcing users to add symbols or capital letters, because users just make predictable tweaks (like Monkey1!). Instead, it recommends making passwords longer.
- CIS Benchmarks: Guidance on standard password lengths and system setup practices.

Software and Environment

- **Kali Linux:** The operating system used to run all Linux terminal commands (bash, md5sum, sha256sum, vi).
- **Hashcat:** The main tool used to crack the target hashes.

Methodology

Lab Setup

All tests were run inside a local Kali Linux terminal. A simple target wordlist named rockyou.txt was created, and an automated script named hash.sh was written to generate hash dumps automatically

1. Wordlist Setup

[rockyou.txt] ---> (bash script / md5 / sha256)

2. Target Hash Files

[target_hashes.txt] | [sha256_target.txt]

3. Hashcat Testing

- Mode 0 (MD5) - Mode 1400 (SHA-256)

- Dictionary Attack - Mask Attack (?d?d?d?d)

- Hybrid Attack - Rule Set (best66.rule)

4. Cracked Results

View unmasked words (--show)

Step-by-Step Plan

1. **Create the Wordlist:** Use vi to create rockyou.txt containing basic test words.
2. **Automate Hashing:** Write hash.sh to convert the words into target hashes.
3. **Test 1 (Dictionary Attack):** Match target hashes directly against rockyou.txt.
4. **Test 2 (Mask Attack):** Guess a 4-digit numeric PIN without using a wordlist.
5. **Test 3 (Rule Attack):** Use best66.rule to automatically capitalize words and add symbols.
6. **Test 4 (Hybrid Attack):** Combine words from rockyou.txt with a 4-digit number at the end.
7. **Test 5 (SHA-256 Algorithm):** Switch Hashcat mode to handle SHA-256 hashes.

Implementation

Script Setup (hash.sh)

To keep from making hashes by hand, a short Bash script named hash.sh was built:

Bash

```
#!/bin/bash

# Script to convert text words into MD5 hashes

while read pass; do

  echo -n "$pass" | md5sum | cut -d" " -f1

done < rockyou.txt > target_hashes.txt
```

The -n option in echo -n stops Linux from adding a hidden newline character at the end of the word, keeping the hash accurate.

Wordlist Creation (rockyou.txt)

Using the command vi rockyou.txt, a custom text file was made containing these common words:

password, monkey, shadow, 123456, 1984, password123, monkey88, kali_linux.

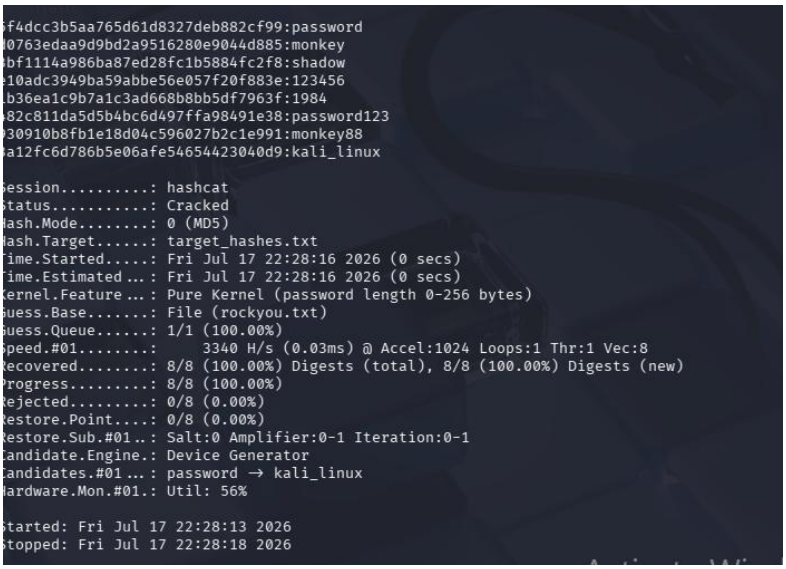
Results and Findings

Test 1: Standard Dictionary Attack (-a 0 -m 0)

The target file was run directly against the custom rockyou.txt dictionary file.

hashcat -m 0 target_hashes.txt rockyou.txt

hashcat -m 0 target_hashes.txt rockyou.txt –show



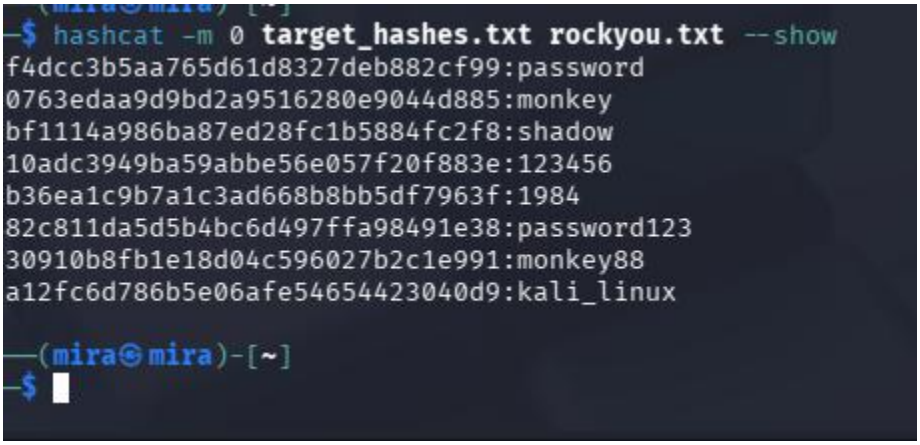


Figure 1: Hashcat instantly cracking simple words using a wordlist.

Test 2: Mask Attack / PIN Guessing (-a 3 -m 0)

A hash for the PIN 1984 was added to the target list (echo -n "1984" | md5sum | cut -d" " -f1 >> target_hashes.txt). The mask ?d?d?d?d told Hashcat to try every 4-digit number combination from 0000 to 9999.

Bash

hashcat -m 0 -a 3 target_hashes.txt ?d?d?d?d

Figure 2: Running a 4-digit mask attack (?d?d?d?d) to find the cracked PIN.

Test 3: Rule Attack (best66.rule)

The password Monkey1! was added to the targets. Since Monkey1! was not typed inside rockyou.txt, a normal dictionary attack could not find it. Using best66.rule allowed Hashcat to try capitalizing words and adding numbers/symbols automatically.

Bash

hashcat -m 0 target_hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best66.rule

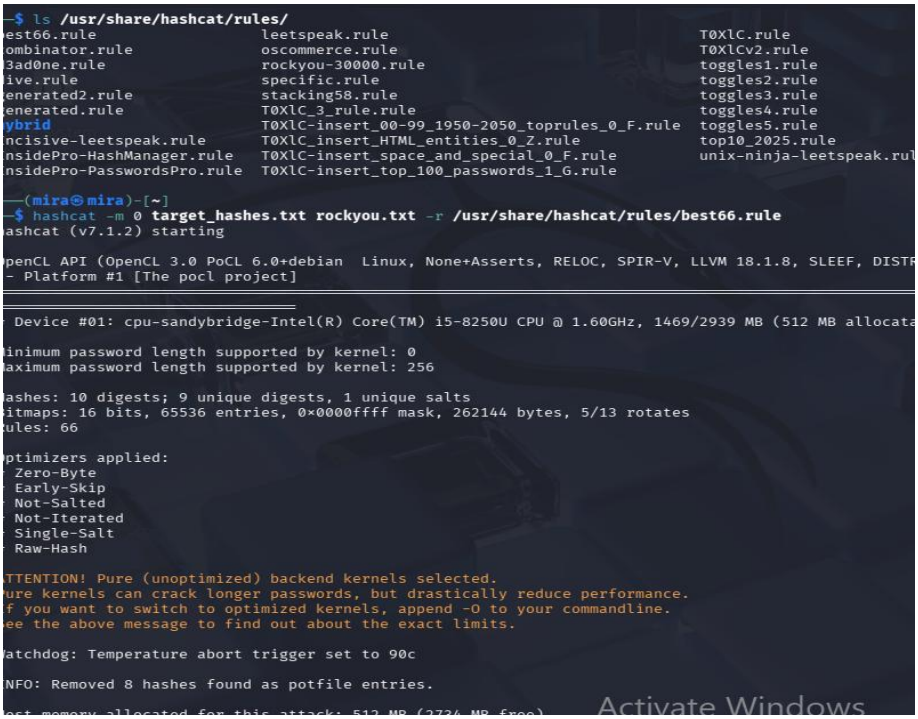
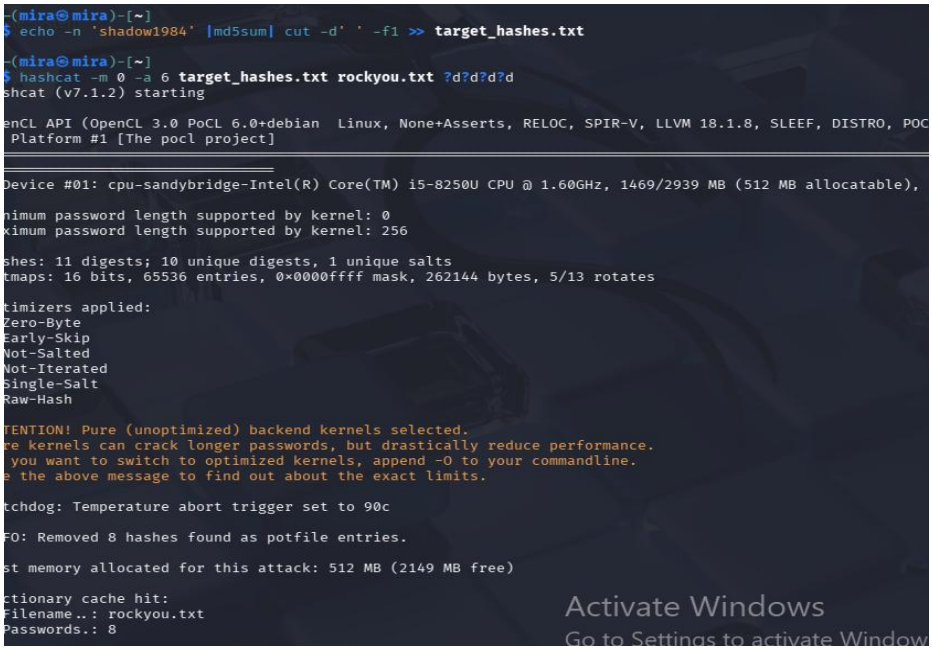


Figure 3: Rule attack using best66.rule to crack modified passwords like Monkey1!.

Test 4: Hybrid Attack (-a 6 -m 0)

A target hash combining a word and a year (shadow1984) was added to the target file. Hybrid Mode (-a 6) took every word in rockyou.txt and attached a 4-digit number mask (?d?d?d?d) onto the end.

Figure 4: Hybrid attack output (-a 6) attaching 4-digit numbers to words to crack shadow1984.



Test 5: SHA-256 Hashing Algorithm (-m 1400)

A SHA-256 hash was created for kali_linux123 (echo -n 'kali_linux123' | sha256sum | cut -d' ' -f1 > sha256_target.txt). Hashcat mode was changed to -m 1400 to handle SHA-256 digests.

Bash

hashcat -m 1400 -a 6 sha256_target.txt rockyou.txt ?d?d?d

hashcat -m 1400 sha256_target.txt --show

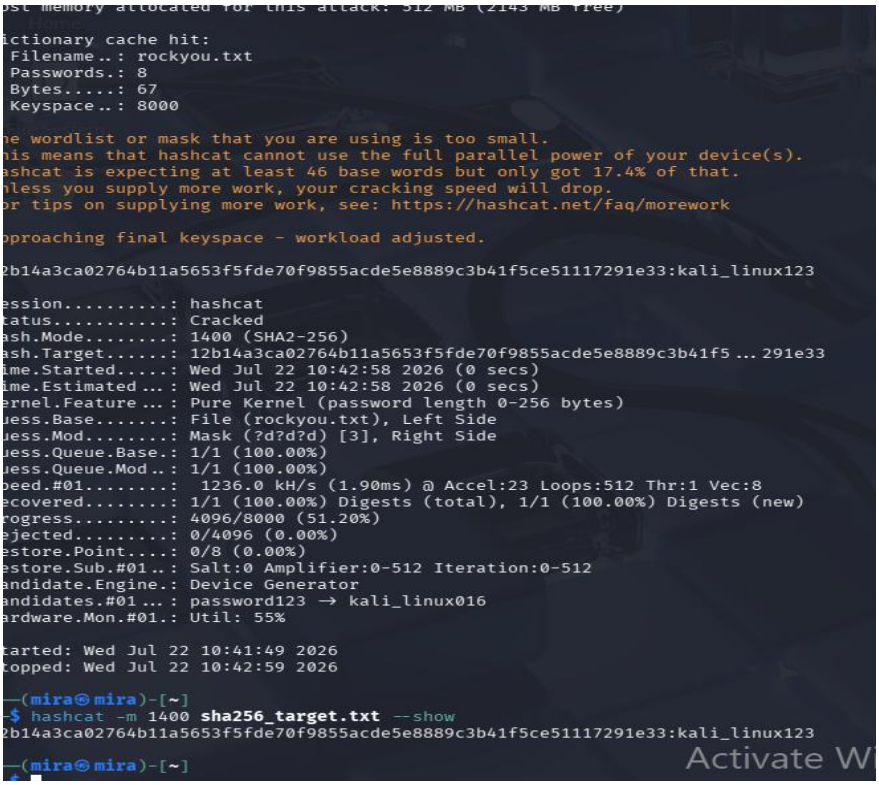


Figure 5: Successfully cracking a SHA-256 target (kali_linux123) using Mode 1400.

Analysis and Recommendations

How Weak Passwords Fail

Password strength depends heavily on how long a password is and how many character types it uses.

Comparison Table

Password	Attack Vector	Search Method	Result Speed	Why it Failed
password	Dictionary (-a 0)	Checks list line-by-line	Instant	Found in standard text files
1984	Mask (-a 3)	Tries 0000 to 9999	Instant	Only 10,000 possible choices
Monkey1!	Rules (best66.rule)	Adds caps and symbols to words	Instant	Simple human substitution pattern
shadow1984	Hybrid (-a 6)	Connects words to 4-digit masks	Instant	Common word mixed with a year
kali_linux123	Hybrid SHA-256 (-m 1400)	Connects words to 3-digit masks	Instant	Unsalted fast hashing setup

Recommendations for Better Security

1. **Focus on Length Instead of Complexity:** Long passphrases made of multiple random words (like blue-sky-running-water-77) are much harder to crack than short complex words like Monkey1!.
2. **Stop Using Fast Hash Functions:** Fast hashing functions like MD5 and SHA-256 allow computers to make billions of guesses a second.

3. **Use Modern Password Storage:** Websites should use slow password-hashing setups like **Argon2** or **bcrypt**, which force the computer to slow down between guesses.
4. **Always Add Salt:** A salt is random extra data added to a password before it is hashed. Salting stops attackers from cracking a huge batch of stolen passwords all at once.

Conclusion

In this lab, five different password cracking techniques were set up and tested in Kali Linux using Hashcat and basic Bash tools. The results show that standard password tricks—like capitalizing letters or adding numbers at the end—do not protect accounts against automated cracking tools. Fast algorithms like MD5 and SHA-256 are too easy for modern computers to process. Systems must use long passphrases and modern, salted password setups to keep account credentials safe.

References

- **NIST (2017).** *Digital Identity Guidelines*. NIST Special Publication 800-63B.
- **MITRE ATT&CK (2024).** *Brute Force: Password Cracking (T1110.002)*.
- **Hashcat Wiki (2026).** *Hashcat User Manual and Rule Syntax*.
- **Stallings, W. (2017).** *Cryptography and Network Security* (7th ed.). Pearson.

Appendices

Appendix A: Target Generation Script (hash.sh)

Bash

```
#!/bin/bash

# Simple loop script to generate target hashes

while read pass; do

  echo -n "$pass" | md5sum | cut -d" " -f1

done < rockyou.txt > target_hashes.txt
```

Appendix B: Output Hash List (cat target_hashes.txt)

Plaintext

```
5f4dcc3b5aa765d61d8327deb882cf99

008d73109218ee1921770cb3e105e263

3bf2d4272199f36f90d1bf430c5e7be1

e10adc3949ba59abbe56e057f20f883e

1223405f63d04d9c733694f4bf78e5f3

5fa24f9a0a030761614050d2432d4365
```